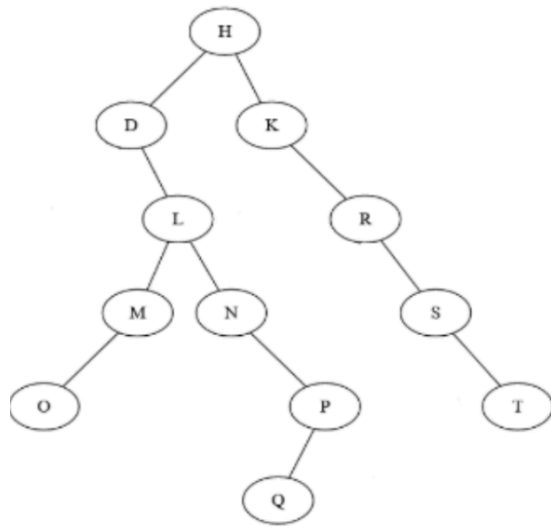


1. nivel usor

Care este ordinea de vizitare a nodurilor din arborele binar dat daca se foloseste Parcurgere in Postordine?



- A. O, M, L, D, Q, P, N, T, S, R, K, H
- B. H, D, L, M, O, N, P, Q, K, R, S, T
- C. H, K, R, S, T, D, L, M, O, N, P, Q
- D. O, M, Q, P, N, L, D, T, S, R, K, H

Raspuns corect : **D**

Justificare raspuns:

Parcurgerea in postordine este o metoda de parcurgere a arborilor binari (sau arborilor in general), in care nodurile sunt vizitate in urmatoarea ordine: **Stanga** → **Dreapta** → **Radacina**

Referinte web: pbinfo.ro

2. nivel mediu

Care este diferenta dintre TAD Multime si TAD Map (Dictionar)?

- A. In multime elementele sunt unice, dar in dictionar putem avea un element cu multiple aparitii.
- B. Multimea poate avea iterator, dar dictionarul nu poate.
- C. Dictionarul contine perechi cheie-valoare, iar multimea elemente simple.
- D. Dictionarul poate fi implementat cu vector dinamic, iar multimea nu poate fi.

Raspuns corect : **C**

Justificare raspuns:

- A. nu este corect deoarece atat multimea, cat si dictionarul au elemente unice
- B. nu este corect deoarece ambele pot avea iterator
- C. definitia
- D. nu este corect deoarece ambele pot fi implementate cu dynamic arrays

Referinte web: cplusplusreference, geeksforgeeks

3. nivel greu

Fie urmatoarea implementare a unui subalgoritm care elimina ultimul element dintr-o Lista Simpla Inlantuita (Single Linked List). Care afirmatie este adevarata despre removeLast?

```
#include <iostream>
using namespace std;
struct SLL {
    static const int MAX_SIZE = 100;
    int next[MAX_SIZE];
    int head;
    int firstEmpty;
    SLL() {
        head = -1;
        firstEmpty = 0;
        for(int i = 0; i < MAX_SIZE - 1; i++)
            next[i] = i + 1;
        next[MAX_SIZE - 1] = -1;
    }
};

void removeLast(SLL& sll) {
    if (sll.head != -1) {
        if (sll.next[sll.head] == -1) {
            sll.next[sll.head] = sll.firstEmpty;
            sll.firstEmpty = sll.head;
            sll.head = -1;
        }
        else {
            int current = sll.head;
            while (sll.next[current] != -1)
                current = sll.next[current];
            sll.next[sll.next[current]] = sll.firstEmpty;
            sll.firstEmpty = sll.next[current];
            sll.next[current] = -1;
        }
    }
}
```

Variante:

- A. Codul nu este corect, dar daca inlocuim conditia de la linia 9 cu `sll.next[sll.next[current]] != -1`, va fi corect.
- B. Codul este corect.
- C. Codul nu este corect, dar daca inversam liniile 5 și 6, va fi corect.
- D. Codul nu este corect, dar daca schimbam linia 12 cu `sll.next[current] = sll.firstEmpty`, va fi corect.

Raspuns corect: A

Justificare raspuns:

- A. Daca inlocuim conditia de la linia 9 cu `sll.next[sll.next[current]] != -1`, va fi corect, deoarece astfel ne oprim la penultimul element in loc de ultimul.
- B. Cazul cu un singur element (liniile 3-6): Corect - elimina singurul element si actualizeaza head
- C. Cazul cu mai multe elemente (liniile 8-14):
 - Problema: Linia 9 foloseste conditia `sll.next[current] != -1`, dar aceasta va cauta ultimul element din lista inlantuita
 - Eroarea: Pentru a elimina ultimul element, trebuie sa ne oprim la penultimul element, nu la ultimul
- D. Verificarea listei vide (linia 2): Corect - verifica daca lista nu este vida

Referinte web: [geeksforgeeks](#), [quora](#)

+ varianta corecta removeLast

```
void removeLast(SLL& sll) {
    if (sll.head != -1) { // lista nu e vida
        if (sll.next[sll.head] == -1) { // doar un singur element
            sll.next[sll.head] = sll.firstEmpty;
            sll.firstEmpty = sll.head;
            sll.head = -1;
        }
        else {
            int current = sll.head;
            // ne oprim la penultimul element
            while (sll.next[sll.next[current]] != -1) {
                current = sll.next[current];
            }

            int last = sll.next[current]; // ultimul element
            sll.next[last] = sll.firstEmpty; // adăugăm în lista de spații libere
            sll.firstEmpty = last; // actualizăm firstEmpty
            sll.next[current] = -1; // ștergem legătura spre ultimul
        }
    }
}
```